agentdna-middleware — Postgres schema (db/db.go New())

Every change below runs on every startup. Original tables = first schema (8792c57, 2026-05-29).

LEGEND

original table | table added later

`col` in original CREATE TABLE | `+ col` added by ALTER / later | `~ col` type / key / default changed | `– col` dropped by a migration

→ solid = real FOREIGN KEY | ⇢ dashed = logical link (no FK)

new_admins
original · admins of an org

	did	TEXT PK
	organization_id	TEXT
	api_key	TEXT
	email	TEXT
	agent_count	INT
	intent_count	INT
	threat_count	INT
	total_users	INT
+	name	TEXT joins admins in user_did_set
+	created_at	TIMESTAMPTZ
–	password	TEXT moved to admin server

sessions
added 2026-10 (session cookies, 34c43fe)

	token_hash	TEXT PK, sha256 of cookie
	account_type	TEXT 'user' | 'admin'
~	account_id	TEXT renamed from email
	ip	TEXT
	user_agent	TEXT
	created_at	TIMESTAMPTZ
	last_seen_at	TIMESTAMPTZ
	expires_at	TIMESTAMPTZ

idx (account_type, account_id)
account_id = org_users.email OR admins.did

credentials
added 2026-10-23 (83d6658)

	credential_id	TEXT PK
	status	TEXT CHECK active|flagged|compromised
	reason	TEXT
	updated_by	TEXT
	updated_at	TIMESTAMPTZ

no row = active · matches auth_evidence.credential_id

new_org_users
original · org (dashboard) users

~	did	TEXT was PK; NULL ok, default 'none'
	organization_id	TEXT
~	api_key	TEXT + partial UNIQUE index
	nft_id	TEXT
	name	TEXT (also re-ADDed, no-op)
~	email	TEXT now PRIMARY KEY
	password	TEXT
	policy	TEXT
	agent_count	INT
	intent_count	INT
	threat_count	INT
	agent_access_list	TEXT JSON array as text
	created_at	TIMESTAMPTZ
+	key	TEXT + index
+	email_pk_added	BOOLEAN never read (dead)

FK

user_dids
added 2026-10-09 (3b43349) · all DIDs of a user

~	email	TEXT PK, FK -> org_users.email
~	dids	TEXT[] GIN index, oldest first
	created_at	TIMESTAMPTZ
–	did	TEXT old layout: 1 row per DID

trigger user_dids_normalize: dedupe, drop ''/none/default–id,
reject DID owned by another user (global advisory lock)
fn user_did_set(did) -> every DID of that user / same–name admins
ON UPDATE CASCADE, ON DELETE CASCADE

new_requests
original · deploy/access agent requests

	request_id	TEXT PK
	request_type	TEXT deploy_agent|access_agent
	policy	TEXT
	creator_did	TEXT
	agent_did	TEXT
	agent_name	TEXT
	request_info	TEXT
	organization_id	TEXT
	status	TEXT pending|approved|denied
	created_at	TIMESTAMPTZ

unchanged since first schema

new_agents
original · deployed agents

	did	TEXT PK
	name	TEXT
	deployer_did	TEXT
	organization_id	TEXT
	nft_id	TEXT
	policy	TEXT
	interactions_count	INT
	intent_count	INT
	threat_count	INT
	created_at	TIMESTAMPTZ (also re-ADDed, no-op)
+	revoked	BOOLEAN NOT NULL default false

new_tools
original · tools / apps

~	did	TEXT PK was (did, org_id) -> did
	name	TEXT
~	organization_id	TEXT now NULLable
+	agents_list	TEXT JSON array as text

every boot: delete duplicate dids, DROP PK, ADD PK (did)

apps
added 2026-08-28 (423ed0b)

	did	TEXT PK
	name	TEXT
	created_at	TIMESTAMPTZ

no query reads or writes this table (dead)

intent_block_data
added 2026-06-19 · code removed 086e684

–	(all columns)	

code is gone but no DROP TABLE:
still exists in older databases

new_intents
original · one workflow (id = nftId)

	interaction_id	TEXT PK
	interaction_ids	TEXT JSON array as text
	initiator_did	TEXT + index
	organization_id	TEXT
	started_at	TIMESTAMPTZ
	ended_at	TIMESTAMPTZ
	status	TEXT running|completed
	threat_detected	INT used as bool
	flow_type	TEXT
	executor	TEXT
	chain_depth	INT
+	review_status	TEXT Unreviewed (was Ongoing)
+	initiator_name	TEXT backfilled each boot
–	provenance_req_id	TEXT -> pending_branches
–	provenance_record_id	TEXT

pending_branches
added 2026-09-30 (c93150e)

	req_id	TEXT PK = /tx result.id
	intent_id	TEXT
	payload	TEXT
	created_at	TIMESTAMPTZ

bridge /rubix/v1/tx -> /rubix/v1/signature;
row deleted when /signature resolves

threat_codes
added 2026-09-01 (b35abf0)

	code	INT PK
	title	TEXT
	description	TEXT

seeded: 1001, 2001–2003, 3001–3409, 4001, 9101–9104
ON CONFLICT DO NOTHING -> edited text never updates

new_interactions
original · one hop inside an intent

	interaction_id	TEXT PK
	initiator_did	TEXT
	initiator_name	TEXT
	interacted_to_did	TEXT
	interacted_to_name	TEXT
	type	TEXT filled from block_type
	direction	TEXT
	threat	INT used as bool
	intent_id	TEXT + index
	organization_id	TEXT
	time	TIMESTAMPTZ
+	signature	TEXT + index
+	hash	TEXT
~	raw_data	JSON JSONB -> JSON (exact bytes)
+	message	TEXT
+	threat_id	TEXT
–	block_type	TEXT merged into type
–	provenance_req_id	TEXT
–	provenance_record_id	TEXT

idx (organization_id, time), (initiator_did, interacted_to_did)

threats
added 2026-09-01 (b35abf0)

	id	TEXT PK
	intent_id	TEXT
	interaction_id	TEXT
	time	TIMESTAMPTZ
	threat_code	INT
	message	TEXT

no index on intent_id / interaction_id

auth_evidence
added 2026-09-22 (4a426d9)

	request_id	TEXT PK part 1
	source	TEXT PK part 2
	run_id	TEXT + index
	auth_method	TEXT
	credential_id	TEXT fingerprint only
	identity_id	TEXT NULL = opaque, not 'none'
	auth_status	TEXT
	key_version	TEXT
	destination	TEXT PK part 3
	organization_id	TEXT
	observed_at	TIMESTAMPTZ

PK (request_id, source, destination)

resources
added 2026-10-23 (4a22c68)

	id	TEXT PK = destination
	name	TEXT
	credentials	JSONB {credId:{authMethod,intents}}
	intents	JSONB [intentId, ...]

BackfillResources() runs every boot

POSSIBLE BUGS (verified against code, 2026-11-04)

[HIGH] 1. A revoked user can still log in and keep using the dashboard
Revocation (UpdateUserStatus, handler.go:2354) only flips is_active on the admin server. There is no revoked/is_active column locally,
Login (handler.go:1015) and VerifyLoginOTP (otp_login.go:86) never check it, and existing sessions are not deleted (DeleteAccountSessions exists, unused here).
Fix: store/check status at login + in the session middleware, and call DeleteAccountSessions('user', email, '') on revoke.

[HIGH] 2. user_did_set() merges admins by NAME only (db.go:553)
JOIN new_admins a2 ON a2.name = a1.name, with no organization filter. Two admins both called 'admin' (or the same person in two orgs)
get each other's DIDs, so user–scoped queries return the other admin's agents, intents and threats. Fix: join on a stable admin–server user id + org.

[HIGH] 3. Migration errors are silently ignored
~40 conn.Exec(...) calls drop the error. Example: if ADD PRIMARY KEY (did) on new_tools fails (NULL/duplicate did), the table is left with NO key and
INSERT ... ON CONFLICT (did) (service.go:2594) fails for every tool registration. Same for the UNIQUE api_key index (db.go:465) if duplicates exist. Fix: log/fail on error.

[MED] 4. new_tools primary key is dropped and re-created on EVERY boot (db.go:593-595)
DELETE duplicates + DROP PK + ADD PK each start: full table lock and index rebuild, and a window with no uniqueness. Wrap in an IF NOT EXISTS check like new_org_users does.

[MED] 5. No migration lock: two instances starting together race
Concurrent DDL (DROP/ADD PK, CREATE OR REPLACE FUNCTION, user_dids conversion) can fail with 'tuple concurrently updated' or deadlock.
Fix: take pg_advisory_lock(<const>) at the start of New() and release at the end, or move to a versioned migration tool (goose/golang–migrate).

[MED] 6. Full-table backfills run on every startup
initiator_name UPDATE joining all interactions (db.go:394), review_status UPDATE, user_dids backfill, BackfillResources() (db.go:691). Startup time grows with data.
Fix: run once (versioned migrations) or guard with a cheap WHERE EXISTS.

[MED] 7. Email is the PRIMARY KEY but is case-sensitive
Logins only TrimSpace the email (otp_login.go:95), so 'Hari@x.com' and 'hari@x.com' become two users / a login 'not registered'. Fix: lower() on write+read, or CITEXT / UNIQUE(lower(email)).

[MED] 8. Sentinel DIDs: '', 'none', 'default-id'
new_org_users.did defaults 'none' (CREATE says ''), many users share it. Any query 'WHERE did = $1' called with 'none' matches all DID–less users.
The user_dids trigger filters these, other queries may not. Prefer NULL for 'no DID yet'.

[LOW] 9. No foreign keys except user_dids -> new_org_users
threats, interactions, pending_branches, sessions etc. are linked only logically, so deletes/rollbacks can leave orphans (e.g. threats of a rolled–back intent). threats has no index on intent_id/interaction_id.

[LOW] 10. Dead / redundant schema
apps table never queried; email_pk_added never read; intent_block_data still exists in old DBs (no DROP TABLE); ADD COLUMN name / created_at are no–ops (already in CREATE).
threat_codes seed uses ON CONFLICT DO NOTHING, so edited titles/descriptions never reach existing DBs.

[LOW] 11. Mixed JSON storage and bool-as-int
interaction_ids, agents_list, agent_access_list are JSON in TEXT (no validation, not indexable) while resources uses JSONB; threat / threat_detected are INT used as bool.

[LOW] 12. Connection pool not configured; log.Fatal inside the library
New() never sets SetMaxOpenConns/SetConnMaxLifetime (txnsim does), so a traffic spike can exhaust Postgres max_connections. log.Fatal kills the process with no cleanup.